
Campus Party Quest

캠퍼스 게임 매칭 애플리케이션

3. Design



제출일	2026.06.05	전공	컴퓨터공학전공
과목	오픈소스 SW 설계	학번	22413558
담당교수	서영석 교수님	이름	김선주

[Revision history]

Revision date	Version #	Description	Author
2026.05.08	1.0	First Writing	김선주
2026.06.05	1.1	5.2 소프트웨어 요구사항 갱신	김선주

[Contents]

1. Introduction	4
2. Class diagram	5
3. Sequence diagram	8
4. State machine diagram	15
5. Implementation requirements	17
6. Glossary	18
7. References	19

1. Introduction 서론

1.1 Summary 요약

Campus Party Quest 는 캠퍼스 환경에서 게임 플레이어 간의 매칭을 체계적으로 지원하는 모바일 애플리케이션이다. 본 문서는 Conceptualization 및 Analysis 단계에서 도출된 요구사항을 바탕으로 구현 가능한 수준의 구조를 정의하는 Design 문서이다. 클래스 다이어그램, 시퀀스 다이어그램, 상태 패턴 다이어그램, 구현 요구사항의 4 가지 설계 산출물을 포함한다. 해당 산출물을 기반으로 시스템의 내부 설계와 동작 흐름을 기술한다.

항목	내용 요약
클래스 다이어그램	6 개 도메인 클래스의 속성·메서드·관계를 정의해 시스템 구조를 명세
시퀀스 다이어그램	11 개 유즈케이스별 객체 간 메시지 흐름으로 동작 시나리오를 기술
상태 머신 다이어그램	Party·Quest 객체의 상태 전이를 정의해 생명주기를 표현
구현 요구사항	시스템 동작에 필요한 하드웨어·소프트웨어 조건 정의

1.2 핵심 설계 포인트

Trust Score 시스템 (어뷰징 방지 핵심 메커니즘)

레퍼런스 게임 기반 매칭 프로세스 (당근마켓 + 크아 조합)

게이미피케이션 (XP/포인트/랭크)

모듈화 구조 명시

2. Class Diagram design 클래스 다이어그램 설계

2.1 Class Diagram 클래스 다이어그램



[그림 2.1-1] Class Diagram

2.2 Class Diagram Description 클래스 다이어그램 상세

1) User

시스템의 핵심 액터. 파티 생성·참여·퀘스트 수행의 주체이며 신뢰도(trustScore)로 신뢰성을 관리한다.

이름	구분	설명
userId	속성	시스템 내 고유 식별자
username	속성	사용자 표시 이름
trustScore	속성	노쇼·어뷰징 방지용 신뢰도 점수. 퀘스트 완료 시 증가, 위반 시 감소
register()	연산	신규 계정 등록. 성공 여부를 bool 로 반환
login()	연산	인증 처리. 성공 여부를 bool 로 반환
updateTrust()	연산	delta 값만큼 trustScore 를 가감

2) Party

게임 파티 정보를 관리하는 클래스. 모집 → 준비 → 매칭 완료의 상태 전이를 담당한다.

이름	구분	설명
partyId	속성	파티 고유 식별자
gameTitle	속성	모집 대상 게임 이름
status	속성	파티 상태 (OPEN / READY / MATCHED). PartyStatus 열거형
create()	연산	새 파티를 생성하고 Party 객체를 반환
cancel()	연산	파티를 취소하고 참여 멤버에게 알림
isFullyReady()	연산	모든 PartyMember 의 isReady 가 true 인지 확인

3) PartyMember

User 와 Party 의 다대다 관계를 표현하는 중간 클래스. 개별 멤버의 준비 상태를 관리한다.

이름	구분	설명
user	속성	참여한 User 객체 참조
party	속성	소속된 Party 객체 참조
isReady	속성	레디 체크 상태. Party.isFullyReady() 판단 기준
setReady()	연산	isReady 를 true 로 변경
cancelReady()	연산	isReady 를 false 로 변경
leave()	연산	파티에서 탈퇴. Party 멤버 목록에서 제거

4) Match

파티 전원 레디 완료 후 생성되는 매칭 확정 객체. Quest 생성의 전제 조건이다.

이름	구분	설명
matchId	속성	매칭 고유 식별자
confirmedAt	속성	매칭 확정 일시
confirm()	연산	매칭을 확정하고 confirmedAt 을 기록
notifyAll()	연산	매칭 결과를 전체 PartyMember 에게 알림
expire()	연산	시간 초과 시 매칭을 무효 처리

5) Quest

매칭 확정 후 생성되는 게임 플레이 단위. 완료 확인을 다수결로 처리해 어뷰징을 방지한다.

이름	구분	설명
questId	속성	퀘스트 고유 식별자
status	속성	퀘스트 상태 (IN_PROGRESS / COMPLETED / ABORTED). QuestStatus 열거형
complete()	연산	status 를 COMPLETED 로 변경하고 Reward 생성 트리거
confirmByMajority()	연산	멤버 과반수 완료 확인 시 complete() 호출
abort()	연산	퀘스트를 강제 종료, trustScore 감점 트리거

6) Reward <abstract>

보상 처리의 공통 인터페이스를 정의하는 추상 클래스.

apply()와 getAmount()를 서브클래스가 각각 구현한다 (다형성).

이름	구분	설명
rewardId	속성	보상 고유 식별자
user	속성	보상을 받는 User 객체 참조
quest	속성	보상이 발생한 Quest 객체 참조
apply()*	추상	보상을 실제로 적용. 서브클래스에서 구체적으로 구현
getAmount()*	추상	보상 수치를 반환. 서브클래스마다 다른 값 반환 (다형성)

7) XPReward

Reward 의 서브클래스. 경험치(XP) 보상을 처리한다. apply()는 User 의 xp 를 xpAmount 만큼 증가시킨다.

이름	구분	설명
xpAmount	속성	지급할 경험치 수치
apply()	연산	User.xp += xpAmount 처리
getAmount()	연산	xpAmount 반환

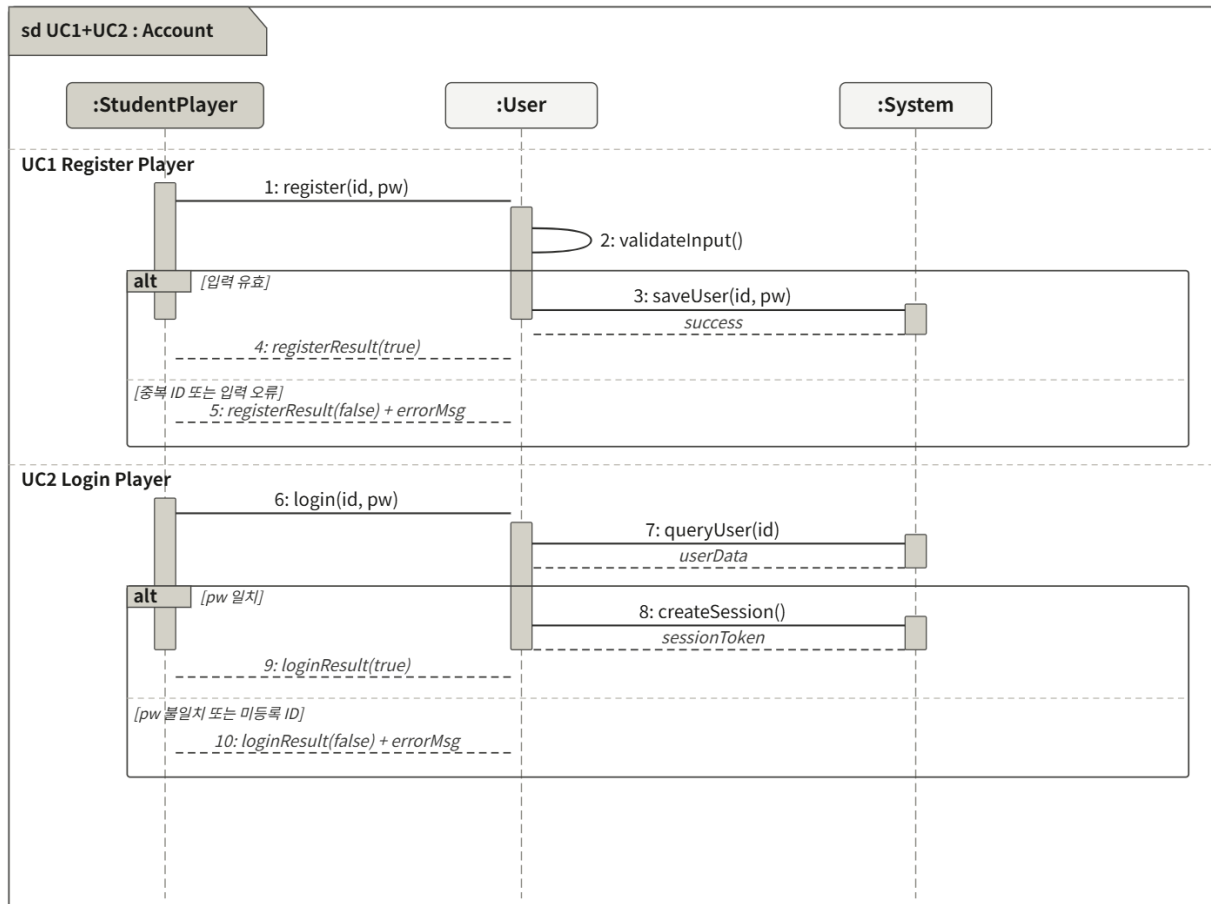
8) PointReward

Reward 의 서브클래스. 포인트 보상을 처리한다. apply()는 User 의 point 를 pointAmount 만큼 증가시킨다.

이름	구분	설명
pointAmount	속성	지급할 포인트 수치
apply()	연산	User.point += pointAmount 처리
getAmount()	연산	pointAmount 반환

3. Sequence diagram 시퀀스 다이어그램

3.1 계정



[그림 3.1-1] Sequence Diagram : UC1 + UC2

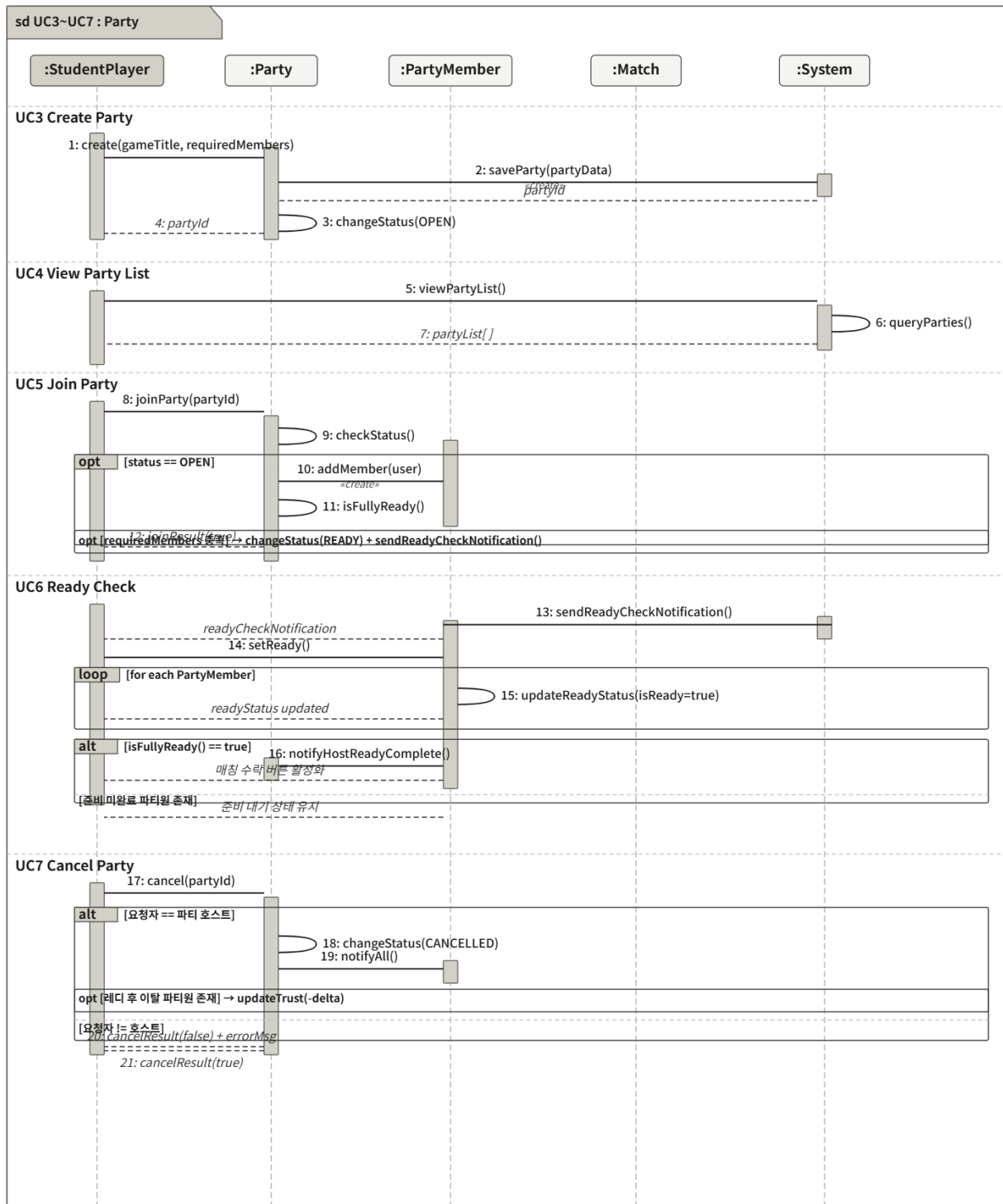
1) UC1 Register

StudentPlayer 가 `register(id, pw)`를 호출하면 User 객체가 `validateInput()`으로유효성을 자체 검사한다. 유효하면 System 에 `saveUser()`를 호출해 저장하고 성공을 반환한다. 중복 ID 이거나 입력 오류면 `registerResult(false)`와 오류 메시지를 반환한다.

2) UC2 Login

StudentPlayer 가 `login(id, pw)`를 호출하면 User 객체가 System 에 `queryUser(id)`로사용자 정보를 조회한다. 비밀번호가 일치하면 `createSession()`으로 세션을 생성하고 `sessionToken` 을 발급해 `loginResult(true)`를 반환한다. 불일치 시 `loginResult(false)`를 반환한다.

3.2 파티



[그림 3.2-1] Sequence Diagram : UC3 – UC7

3) UC3 Create Party

StudentPlayer 가 create(gameTitle, requiredMembers)를 호출하면 Party 객체가 System 에 saveParty()로 저장하고 partyId 를 반환받는다. 이후 changeStatus(OPEN)으로 모집 상태를 전환한다.

4) UC4 View Party

List StudentPlayer 가 viewPartyList()를 호출하면 System 이 queryParties()로 DB 를 조회해 파티 목록 배열을 반환한다.

5) UC5 Join Party

StudentPlayer 가 joinParty(partyId)를 호출하면 Party 가 checkStatus()로 상태를 확인한다.

opt [status == OPEN] 조건에서만 addMember(user)로 PartyMember 를 생성하고, isFullyReady()로 인원 충족 여부를 확인해 충족 시 changeStatus(MATCHED)를 호출한다.

6) UC6 Ready Check

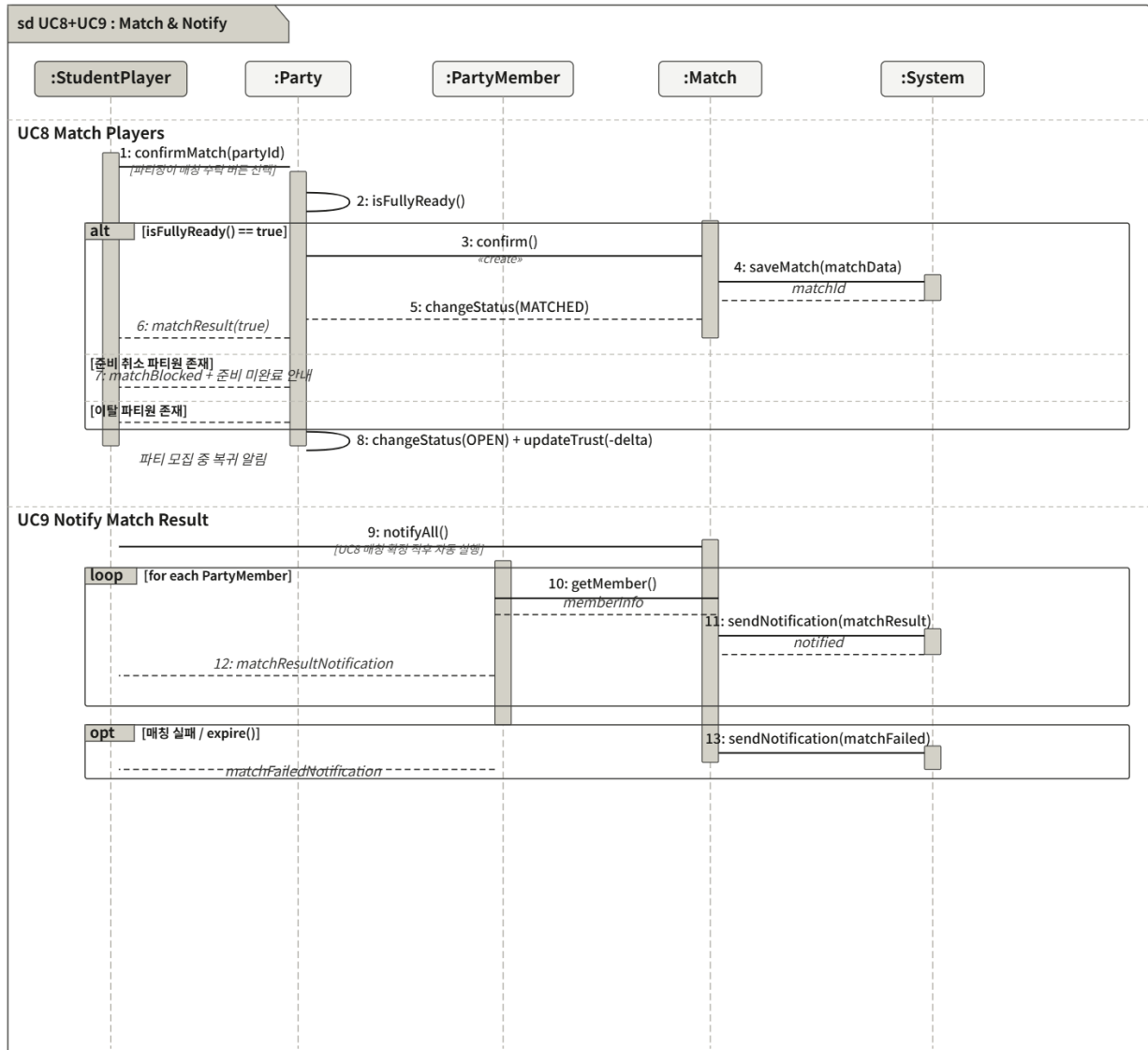
신규 추가 System 이 전원에게 준비 요청 알림 발송 → 각 파티원 setReady() → loop 으로 상태 갱신 → alt [전원 레디] 시 파티장에게 매칭 수락 버튼 활성화 알림.

7) UC7 Cancel Party

cancel(partyId) 호출 시 alt 로 요청자가 호스트인지 분기한다.

호스트면 changeStatus(CANCELLED) 후 notifyAll()로 전체 멤버에게 알림을 보낸다. 호스트가 아니면 cancelResult(false)를 반환한다.

3.3 매칭



[그림 3.3-1] Sequence Diagram : UC8 + UC9

UC8 Match Players

파티장이 매칭 수락 버튼을 선택하면 confirmMatch(partyId)가 호출된다. Party 객체가 isFullyReady()로 전원 레디 상태를 최종 검증한다.

alt [isFullyReady() == true] 조건에서 confirm()으로 Match 객체를 생성하고 System 에 saveMatch()로 저장 후 matchId 를 반환받는다. 이후 changeStatus(MATCHED)로 파티 상태를 전환하고 matchResult(true)를 반환한다.

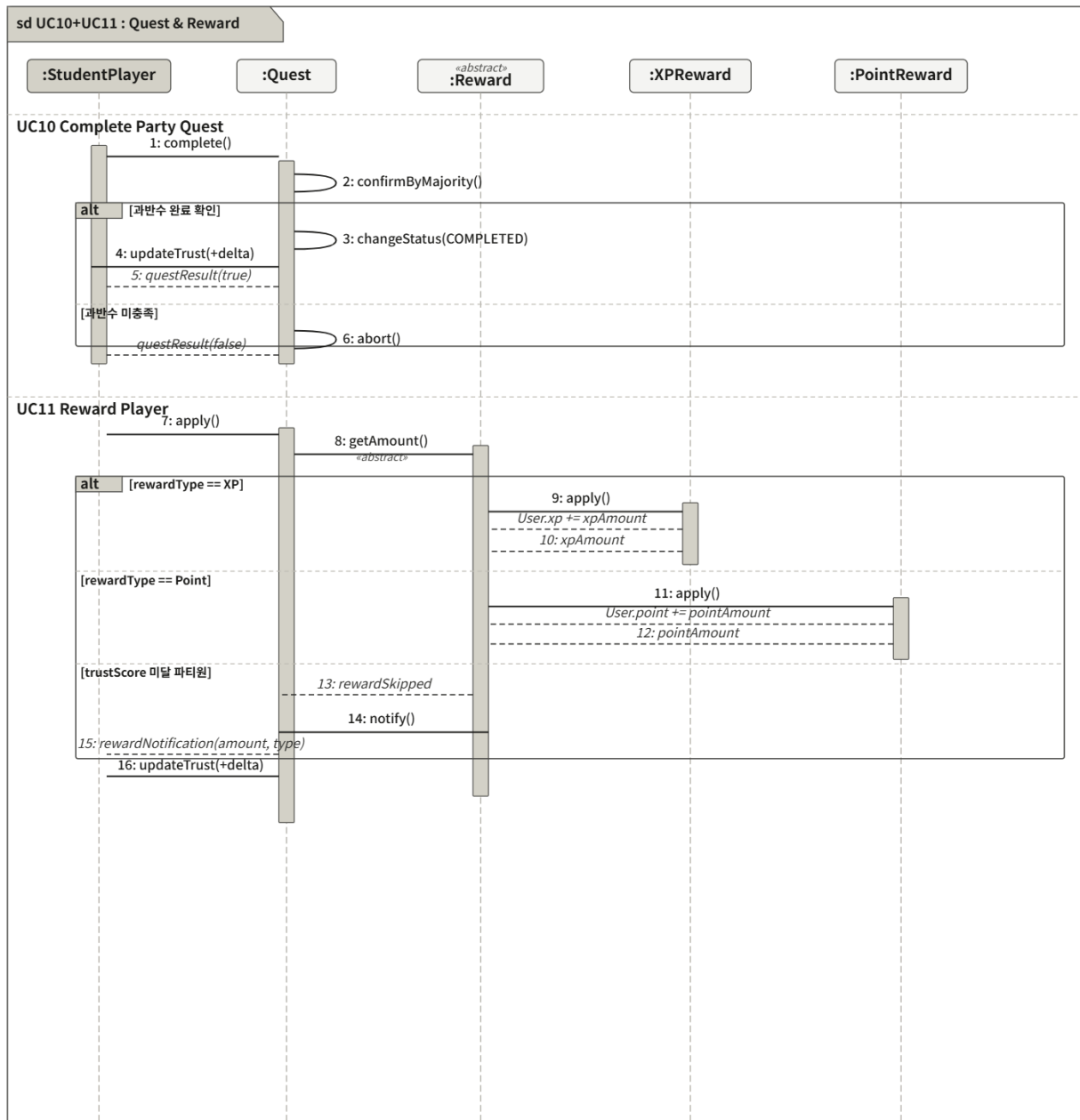
준비 취소 파티원이 있으면 수락을 차단하고 안내 메시지를 반환한다. 이탈 파티원이 있으면 changeStatus(OPEN)으로 파티를 모집 중으로 되돌리고 updateTrust(-delta)로 신뢰도를 차감한다.

UC9 Notify Match Result

UC8 매칭 확정 직후 자동 실행된다. Match 객체가 notifyAll()을 호출하면 loop [for each PartyMember]로 전체 멤버를 순회하며 getMember()로 멤버 정보를 가져온다.

System 의 sendNotification(matchResult)으로 각 멤버에게 알림을 전송하고 matchResultNotification 을 반환한다. opt [매칭 실패 / expire()] 조건에서는 sendNotification(matchFailed)를 별도 전송하고 matchFailedNotification 을 반환한다.

3.4 퀘스트/보상



[그림 3.4-1] Sequence Diagram : UC10 + UC11

UC10 Complete Party Quest

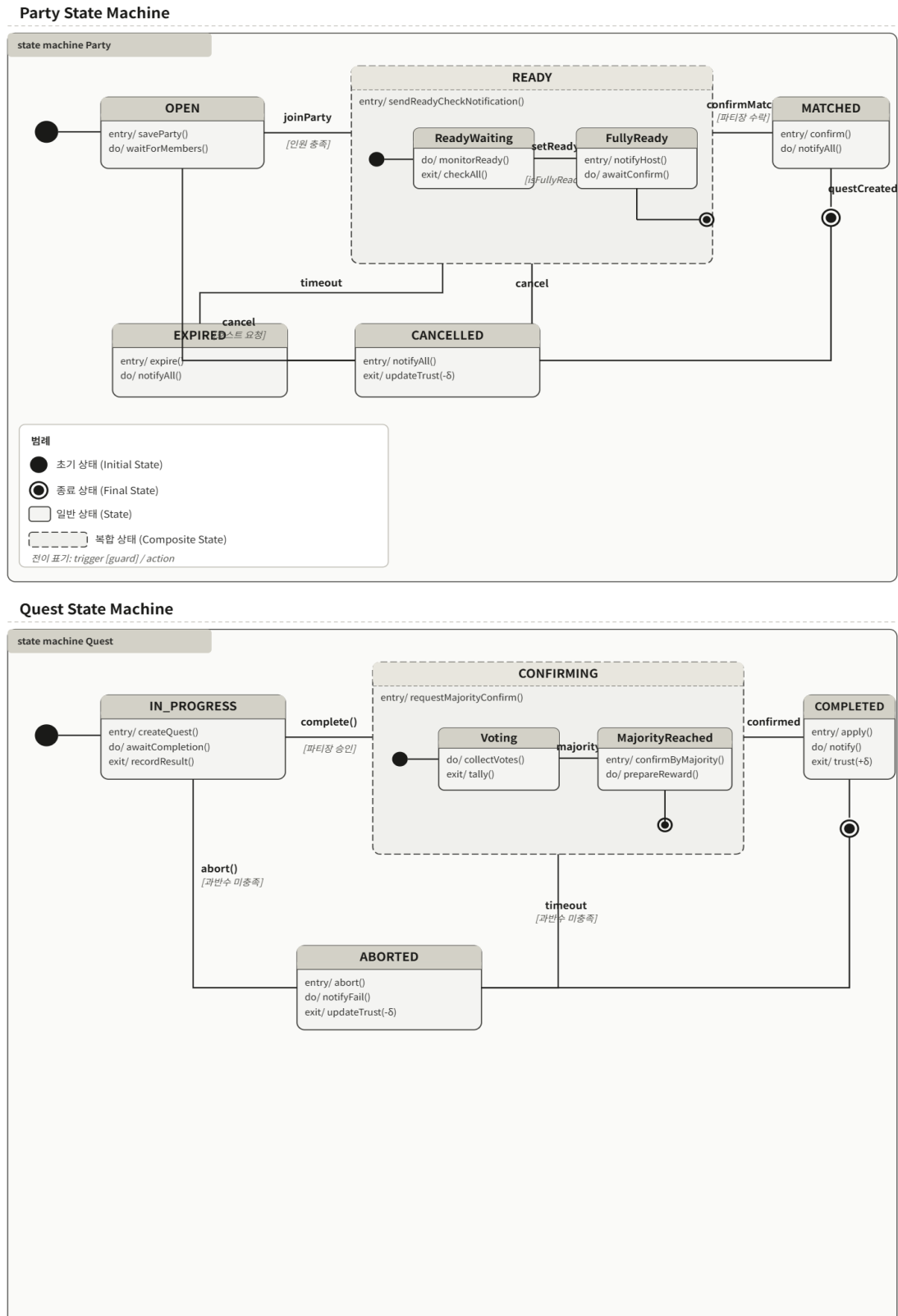
StudentPlayer 가 complete()을 호출하면 Quest 가 confirmByMajority()로 멤버 과반수 확인을 자체 검사한다. alt [과반수 완료 확인] 조건에서 changeStatus(COMPLETED)로 상태를 전환하고 updateTrust(+delta)로 신뢰도를 상승시킨 후 questResult(true)를 반환한다.

과반수 미충족 시 abort()로 강제 종료하고 questResult(false)를 반환한다.

UC11 Reward Player Quest

완료 후 apply()가 호출되면 Reward 추상 클래스가 getAmount()로 보상 수치를 요청한다. alt [rewardType == XP] 분기에서 XPReward.apply()가 User.xp += xpAmount 를 처리하고, [rewardType == Point] 분기에서 PointReward.apply()가 User.point += pointAmount 를 처리한다.
trustScore 미달 파티원은 보상에서 제외되고, 처리 완료 후 notify()로 보상 알림을 전송한다.

4. State machine diagram 상태 머신 다이어그램



Party State Machine

Party 객체는 OPEN → READY → MATCHED 의 메인 흐름을 가진다. READY 는 ReadyWaiting → FullyReady 두 개의 서브상태를 갖는 복합상태로, 전원 레디 완료 시 호스트에게 알림이 전송된다. 파티장이 confirmMatch 로 수락하면 MATCHED 로 전이한다. 어느 상태에서든 cancel 이벤트 발생 시 CANCELLED 로, 레디체크 시간 초과 시 EXPIRED 로 전이하며 모두 최종 상태로 종료된다.

Quest State Machine

Quest 객체는 IN_PROGRESS → CONFIRMING → COMPLETED 의 메인 흐름을 가진다. CONFIRMING 은 Voting → MajorityReached 두 개의 서브상태를 갖는 복합상태로, 과반수 투표 수집 후 confirmByMajority()가 실행된다. 과반수 미충족 또는 타임아웃 시 ABORTED 로 전이하며 신뢰도 차감이 발생한다. COMPLETED 시 보상 지급(apply()) 및 신뢰도 증가가 실행된다.

5. Implementation requirements 구현 요구사항

5.1 Hardware Requirements 하드웨어 요구사항

용어	최소 사양	권장 사양
CPU	Quad-core 1.8GHz	Octa-core 2.4GHz 이상
RAM	2GB	4GB 이상
저장공간	100MB 여유 공간	500MB 이상
네트워크	Wi-Fi / LTE	Wi-Fi 5GHz 또는 5G
디스플레이	5.0 인치 HD (720p)	6.0 인치 FHD (1080p) 이상

5.2 Software Requirements 소프트웨어 요구사항

항목	내용
운영체제	Android 10.0 이상 / iOS 14.0 이상
데이터베이스	Firebase Firestore 또는 MySQL
인증	Firebase Authentication (세션 토큰 기반)
알림	Firebase Cloud Messaging (FCM)
개발 IDE	Android Studio / VS Code
버전 관리	Git / GitHub

6. Glossary 용어 정리

용어	설명
파티 (Party)	함께 게임을 플레이하기 위해 구성된 플레이어 그룹. 파티장이 생성하며 모집 완료 후 게임을 진행한다.
파티장 (Party Host)	파티를 생성한 사용자. 매칭 수락 및 퀘스트 완료 최종 승인 권한을 가진다.
파티원 (Party Member)	파티에 참여한 일반 사용자. 레디 및 완료 확인에 참여한다.
준비/레디 (Ready)	게임 플레이 준비 완료를 의미하는 상태. 파티원 전원이 레디를 완료해야 파티장이 매칭을 수락할 수 있다.
매칭 (Matching)	파티원 전원 레디 완료 후 파티장이 수락하여 파티가 확정되는 과정.
퀘스트 (Quest)	매칭 완료 후 생성되는 게임 플레이 단위. 실제 게임 플레이 후 파티장이 완료를 승인하면 종료된다.
XP (Experience Points)	퀘스트 완료 시 지급되는 경험치. 플레이어의 활동 누적 지표로 활용된다.
포인트 (Point)	퀘스트 완료 시 XP 와 함께 지급되는 보상.
신뢰도 점수 (Trust Score)	플레이어의 매너 및 참여 성실도를 나타내는 수치. 노쇼 또는 부정 평가 누적 시 차감되며 일정 수준 이하 시 파티 참여가 제한된다.
노쇼 (No-show)	매칭 확정 후 실제 게임 플레이에 참여하지 않는 행위. 신뢰도 점수 차감 대상이다.
준비/레디 체크 (Ready Check)	파티원 전원의 준비 상태를 확인하는 단계. 전원 레디 완료 시 파티장의 매칭 수락이 가능해진다.
파티 퀘스트 (Party Quest)	여러 파티원이 함께 수행하는 게임 기반 활동. 매칭 완료 후 자동 생성되며 완료 시 보상이 지급된다.

7. References 참고문헌

1. 이전 문서 참조

- 본인 작성 Conceptualization 문서 (1. Conceptualization, 22413558 김선주) 내용 기반